

Prelim Lab Activity #2

Constants.sol

The screenshot displays the Remix IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' panel shows the deployment of the 'Constants' contract. The 'VALUE' field is set to 0, and the 'CONTRACT' is 'Constants - contracts/Constants.sol'. The 'Deploy' button is highlighted. Below this, the 'Deployed Contracts' section shows the contract's address and variables: 'MY_ADDRESS' (0x777788889999AaAbBbCcCcdDdDeeEFFFfCccC), 'MY_UINT' (123), and 'Low level interactions' (CALLDATA). The main editor shows the 'Constants.sol' contract code, which defines two constants: 'MY_ADDRESS' and 'MY_UINT'. The bottom panel shows the 'Explain contract' section with a list of transactions and their details, including gas usage and return values.

ErrorHandling.sol

The screenshot displays the Remix IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' panel shows the deployment of the 'ErrorHandling' contract. The 'VALUE' field is set to 0, and the 'CONTRACT' is 'ErrorHandling - contracts/ErrorHandl'. The 'Deploy' button is highlighted. Below this, the 'Deployed Contracts' section shows the contract's address and variables: 'setvalue' (0), 'getvalue', 'valueMapping', and 'Low level interactions' (CALLDATA). The main editor shows the 'ErrorHandling.sol' contract code, which defines a mapping 'valueMapping' and functions 'setvalue' and 'getvalue'. The bottom panel shows the 'Explain contract' section with a list of transactions and their details, including gas usage and return values. A red error message is visible in the logs, indicating a revert due to a value not being set.

Mapping.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract MappingExample {
5     // Define a mapping to store values associated with addresses
6     mapping(address => uint256) public valueMapping;
7
8     // Function to set a value for the sender's address
9     function setValue(uint256 _value) public {
10         valueMapping[msg.sender] = _value;
11     }
12
13     // Function to retrieve the value associated with the sender's address
14     function getValue() public view returns (uint256) {
15         return valueMapping[msg.sender];
16     }
17 }
```

Deployed Contracts

Balance: 0 ETH

setValue 10

getValue

uint256 10

valueMapping address

Low level interactions

CALLDATA

Transaction

Explain contract

AI copilot

Listen on all transactions

Filter with transaction hash or address

- [v] from: 0x583...edc4 to: Ownable.constructor value: 0 wei data: 0x088...e083 logs: 0 hash: 0x2ed...d4df creation of MappingExample pending...
- [v] from: 0x583...edc4 to: MappingExample.constructor value: 0 wei data: 0x088...e083 logs: 0 hash: 0x5d...814e transaction to MappingExample.setValue pending...
- [v] from: 0x583...edc4 to: MappingExample.setValue(uint256) 0x5d...90f9 value: 0 wei data: 0x552...0000 logs: 0 hash: 0x6db...17cb7 call to MappingExample.getValue
- [call] from: 0x5830d6a701c568545cf889fcb875f5b6d0c4 to: MappingExample.getValue() data: 0x209...65255

FunctionModifier.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.20;
3
4 contract FunctionModifier {
5     mapping(address => uint256) public valueMapping;
6
7     // Modifier to check if the value being set is not zero
8     modifier nonZeroValue(uint256 _value) {
9         require(_value != 0, "Value cannot be zero");
10         _;
11     }
12
13     // Modifier to check if there's a value set for the sender
14     modifier valueHasBeenSet() {
15         require(valueMapping[msg.sender] != 0, "No value set for sender.");
16         _;
17     }
18
19     // Function to set a value for the sender's address
20     function setValue(uint256 _value) public nonZeroValue(_value) {
21         valueMapping[msg.sender] = _value;
22     }
23
24     // Function to retrieve the value associated with sender's address
25     function getValue() public view valueHasBeenSet returns (uint256) {
26         return valueMapping[msg.sender];
27     }
28 }
```

Deployed Contracts

Balance: 0 ETH

setValue 5

getValue

uint256 5

valueMapping address

Low level interactions

CALLDATA

Transaction

Explain contract

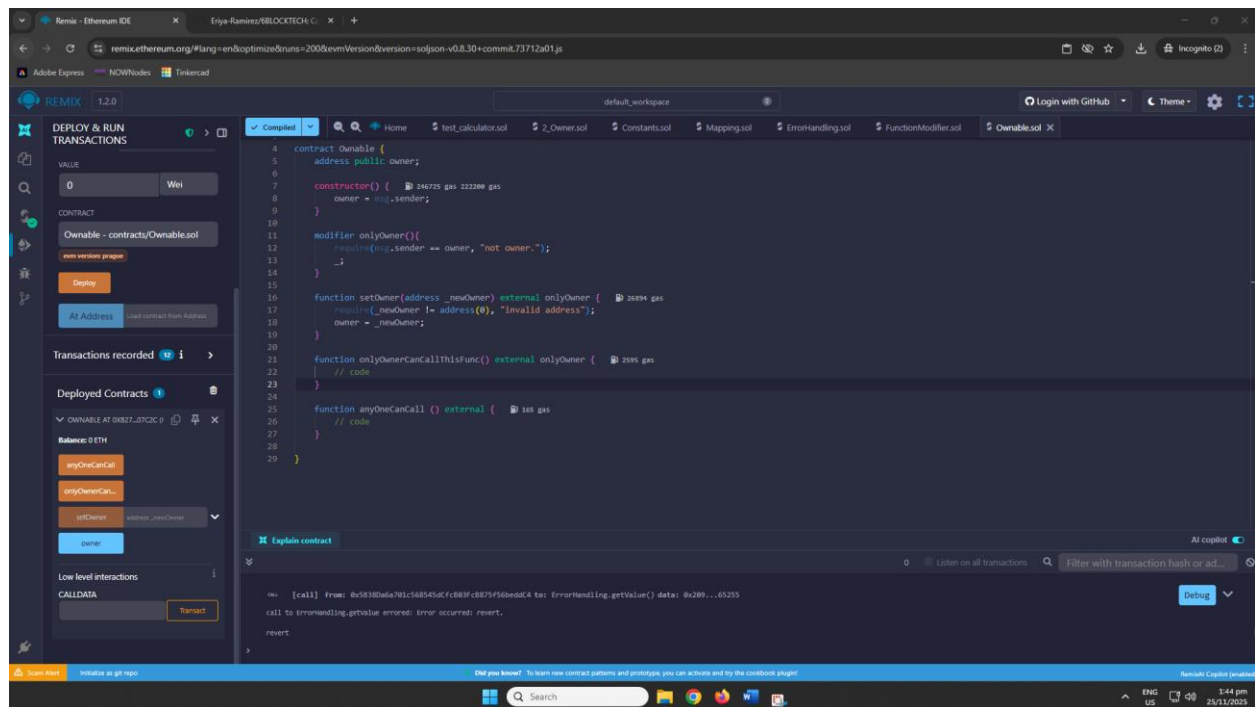
AI copilot

Listen on all transactions

Filter with transaction hash or address

- [v] from: 0x583...edc4 to: FunctionModifier.setValue(uint256) 0x5d...a5692 value: 0 wei data: 0x552...0000 logs: 0 hash: 0x3f...179f8 call to FunctionModifier.getValue
- [call] from: 0x5830d6a701c568545cf889fcb875f5b6d0c4 to: FunctionModifier.getValue() data: 0x209...65255

Ownable.sol (with constructor)



REFLECTION QUESTIONS

- **Function Modifiers & Ownable**

When should you use a modifier like onlyOwner instead of inline checks, and what risks arise if ownership isn't managed properly?

- *When the same ownership restriction applies to several functions, use 'onlyOwner' modifiers instead of inline checks; this keeps code DRY, reduces human error, and makes access control crystal clear at a look. Modifiers are cleaner and less prone to errors (for example, neglecting to check in one function). However, if ownership is never properly transferred or renounced, the deal may become permanently locked under a dead, compromised, or lost private key; a single point of failure that can prevent upgrades, withdrawals, or emergency actions indefinitely. Poor ownership management is one of the leading causes of failed multimillion-dollar contracts.*

- **Error Handling**

How do you choose between require, revert, and assert, and why might custom errors be better than error strings?

- *In Solidity, use 'require' to validate inputs and expected errors (it's inexpensive and permits custom errors), 'revert' when you need precise*

control inside complex logic, and 'assert' only for invariants that should never fail, because 'assert' consumes all remaining gas if triggered. Custom errors outperform string messages by saving up to 90% gas and displaying neatly in wallets and tools. For everyday checks, use 'require' with custom errors—it's the cheapest, safest, and most readable option.

- **Constants & Variables**

When should a value be constant, immutable, or mutable, and how does that choice affect gas cost and flexibility?

- *Use '**constant**' for values that are fixed permanently at compile time (such as PI or a token name) as it requires no gas to read and are baked into bytecode. Use '**immutable**' for values set once in the constructor (such as owner or maximum supply) they also require no gas to read but allow for one-time setup. Use **normal mutable variables** only if the value changes frequently. Choosing 'constant' or 'immutable' saves enormous deployment and runtime gas while slightly lowering flexibility, but it also avoids costly mistakes and keeps your contract cheaper and safer.*